DBMS Lab

Complete Query Cheat Sheet

Topics 2–10 · MySQL & MongoDB

Topic 2 — DDL: Data Definition Language

CREATE TABLE — Creates a new table with constraints

Define columns, data types, and apply PRIMARY KEY, UNIQUE, NOT NULL, CHECK, FOREIGN KEY constraints.

```
CREATE TABLE Department (
    Dept_ID   INT PRIMARY KEY,
    Dept_Name VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE Student (
    Stud_ID   INT PRIMARY KEY,
    Stud_Name VARCHAR(50) NOT NULL,
    Age       INT CHECK (Age >= 18),
    Email     VARCHAR(100) UNIQUE,
    Dept_ID   INT,
    FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
);
```

ALTER TABLE — Modify an existing table structure

Add columns, change data types, rename columns, set/drop defaults.

```
ALTER TABLE Student ADD City VARCHAR(30);
ALTER TABLE Student MODIFY Stud_Name VARCHAR(100) NOT NULL;
ALTER TABLE Student RENAME COLUMN City TO Location;
ALTER TABLE Student ALTER COLUMN Location SET DEFAULT 'Unknown';
```

RENAME TABLE — Rename an existing table

```
RENAME TABLE Student TO Student_Details;
```

TRUNCATE — Remove all rows but keep table structure

Faster than DELETE with no WHERE; cannot be rolled back in most engines.

```
TRUNCATE TABLE Student_Details;
```

DROP TABLE — Permanently delete a table and all its data

```
DROP TABLE Student_Details;
```

FOREIGN KEY violation — What happens when constraint fails

```
INSERT INTO Student_Course (Student_ID, Course_ID, Enrollment_Date)
VALUES (1, 999, '2023-10-01');
-- ERROR 1452: Cannot add or update a child row: foreign key constraint fails
-- Course_ID 999 does not exist in the Course table
```

Topic 3 — DML, DCL & TCL Commands

DML — Data Manipulation Language

INSERT — Add rows into a table

```
INSERT INTO Book (Book_ID, Book_Name, Author, Price, Category) VALUES
(1, 'Database Systems', 'Elmasri', 500.00, 'Education'),
(2, 'Data Structures',  'Sahni',   350.00, 'Programming');
```

SELECT — Read / query data from a table

*Use * for all columns, or list specific columns. Add WHERE, LIKE, AND/OR for filtering.*

```
SELECT * FROM Book;
SELECT Book_Name, Price, Price + 50.00 AS New_Price FROM Book;
SELECT * FROM Book WHERE Category = 'Education' AND Price > 450;
SELECT * FROM Book WHERE Book_Name LIKE 'D%';   -- starts with D
```

UPDATE — Modify existing rows

```
UPDATE Book SET Price = Price * 1.10 WHERE Category = 'Programming';
```

DELETE — Remove specific rows

```
DELETE FROM Book WHERE Price < 350.00;
```

Built-in String & Numeric Functions

UPPER/LOWER change case; LENGTH returns character count; SUBSTR extracts part of string; ROUND rounds decimals; NOW() returns current timestamp.

```
SELECT UPPER(Book_Name) AS Book_UPPER, LOWER(Author) AS Author_LOWER FROM Book;
SELECT LENGTH(Book_Name) AS name_length FROM Book;
SELECT SUBSTR(Book_Name, 1, 4) AS short_name FROM Book;
SELECT ROUND(Price, 2) FROM Book;
SELECT NOW();
```

Set Operators — Combine results from multiple queries

UNION returns unique rows from both. UNION ALL includes duplicates. INTERSECT returns only common rows.

```
-- UNION: unique book names from both tables
SELECT Book_Name FROM Book
UNION
SELECT Book_Name FROM Issued_Book;

-- INTERSECT: books present in BOTH tables
SELECT Book_Name FROM Book
INTERSECT
SELECT Book_Name FROM Issued_Book;
```

DCL — Data Control Language

CREATE USER — Add a new database user

```
CREATE USER 'library_user'@'localhost' IDENTIFIED BY 'password123';
```

GRANT — Give privileges to a user

```
GRANT SELECT, UPDATE ON Book TO 'library_user'@'localhost';
```

REVOKE — Remove privileges from a user

```
REVOKE UPDATE ON Book FROM 'library_user'@'localhost';
```

TCL — Transaction Control Language

START TRANSACTION / SAVEPOINT / ROLLBACK / COMMIT

Transactions group multiple operations. SAVEPOINT marks a point to roll back to. COMMIT makes changes permanent.

```
START TRANSACTION;

INSERT INTO Book (Book_ID, Book_Name, Author, Price, Category)
VALUES (6, 'Operating Systems', 'Galvin', 600.00, 'Education');

SAVEPOINT sp1;                          -- save state here

UPDATE Book SET Price = 650.00 WHERE Book_ID = 6;

ROLLBACK TO sp1;                        -- undo UPDATE, keep INSERT

COMMIT;                                 -- finalize (INSERT at 600 saved)
```

Topic 4 — Aggregate Functions & Grouping

COUNT — Count rows per group

```
SELECT Department, COUNT(Student_ID) AS Total_Students
FROM Student_Result
GROUP BY Department;
```

AVG — Average value per group

```
SELECT Subject, AVG(Marks) AS Average_Marks
FROM Student_Result
GROUP BY Subject;
```

MAX / MIN — Highest and lowest values per group

```
SELECT Subject, MAX(Marks) AS Highest_Marks FROM Student_Result GROUP BY Subject;
SELECT Subject, MIN(Marks) AS Lowest_Marks  FROM Student_Result GROUP BY Subject;
```

SUM — Total value per group

```
SELECT Department, SUM(Marks) AS Total_Marks
FROM Student_Result
GROUP BY Department;
```

WHERE — Filter individual rows BEFORE grouping

WHERE runs before aggregation; excludes rows from the calculation entirely.

```
SELECT Department, AVG(Marks) AS Average_Marks
FROM Student_Result
WHERE Marks > 70           -- only rows with Marks > 70 enter the GROUP
GROUP BY Department;
```

HAVING — Filter groups AFTER aggregation

HAVING runs after GROUP BY; filters entire groups based on aggregate results.

```
-- Departments whose total marks exceed 400
SELECT Department, SUM(Marks) AS Total_Marks
FROM Student_Result
GROUP BY Department
HAVING SUM(Marks) > 400;

-- Subjects where average marks > 80
SELECT Subject, AVG(Marks) AS Average_Marks
FROM Student_Result
GROUP BY Subject
HAVING AVG(Marks) > 80;
```

ORDER BY — Sort results ascending or descending

```
SELECT * FROM Student_Result ORDER BY Marks DESC;   -- highest first
SELECT * FROM Student_Result ORDER BY Marks ASC;    -- lowest first
```

Topic 5 — Joins & Views (BankDB)

JOIN Operations

INNER JOIN — Returns only rows with matches in both tables

```
SELECT e.emp_no, e.e_name, e.salary, e.dept, b.bname
FROM employee_master e
INNER JOIN branch_master b ON e.branch_id = b.branch_id
ORDER BY e.emp_no;
```

INNER JOIN with WHERE — Add filter on top of a join

```
SELECT e.emp_no, e.e_name, e.salary, e.dept, b.bname
FROM employee_master e
INNER JOIN branch_master b ON e.branch_id = b.branch_id
WHERE e.dept = 'Admin';
```

Multi-table JOIN — Chain multiple joins together

```
SELECT e.e_name, c.phnno, a.city
FROM employee_master e
INNER JOIN contact_details c     ON e.emp_no = c.emp_id
INNER JOIN emp_address_details a ON e.emp_no = a.emp_id;
```

LEFT JOIN — All rows from left table; NULL if no match on right

```
SELECT e.e_name, c.emailid
FROM employee_master e
LEFT JOIN contact_details c ON e.emp_no = c.emp_id;
-- Employees without contact info appear with NULL emailid
```

SELF JOIN — Join a table with itself (e.g. manager-employee relationship)

```
SELECT e.e_name AS Employee, m.e_name AS Manager
FROM employee_master e
LEFT JOIN employee_master m ON e.manager_id = m.emp_no;
```

DISTINCT — Remove duplicate rows from results

```
SELECT DISTINCT b.bname
FROM employee_master e
INNER JOIN branch_master b ON e.branch_id = b.branch_id
WHERE e.salary > 25000;
```

Views

CREATE VIEW — Save a query as a named virtual table

A view does not store data; it re-runs the query each time it is selected.

```
CREATE VIEW branch_employee_count AS
SELECT b.bname, COUNT(e.emp_no) AS Employee_Count
FROM employee_master e
INNER JOIN branch_master b ON e.branch_id = b.branch_id
GROUP BY b.bname;
```

```
SELECT * FROM branch_employee_count;   -- use the view like a table
```

View with AVG and SUM

```
CREATE VIEW branch_salary_summary AS
SELECT b.bname, AVG(e.salary) AS Avg_Salary, SUM(e.salary) AS Total_Salary
FROM employee_master e
INNER JOIN branch_master b ON e.branch_id = b.branch_id
GROUP BY b.bname;
```

UPDATE through a view — Only works on single-table views

```
CREATE VIEW emp_dept_view AS
SELECT emp_no, e_name, dept FROM employee_master;

UPDATE emp_dept_view SET dept = 'HR' WHERE emp_no = 106;   -- works

-- JOIN views are NOT updatable across multiple base tables:
UPDATE emp_branch_view SET bname = 'Shivaji Nagar' WHERE emp_no = 101;
-- ERROR 1393: Can not modify more than one base table through a join view
```

Topic 6 — Subqueries

IN — Match rows whose value exists in a subquery result

```
SELECT * FROM Customer
WHERE customer_id IN (SELECT customer_id FROM Orders);
```

NOT IN — Match rows whose value does NOT exist in subquery result

```
SELECT * FROM Customer
WHERE customer_id NOT IN (SELECT customer_id FROM Orders);
```

Scalar subquery — Subquery returns a single value (MAX, AVG, etc.)

```
-- Product with the highest price
SELECT * FROM Product
WHERE price = (SELECT MAX(price) FROM Product);

-- Products above average price
SELECT * FROM Product
WHERE price > (SELECT AVG(price) FROM Product);
```

Subquery with JOIN — Filter using a joined subquery

```
-- Customers who ordered a specific product (product_id = 101)
SELECT * FROM Customer
WHERE customer_id IN (
    SELECT customer_id FROM Orders WHERE product_id = 101
);

-- Customers who ordered products costing more than 20000
SELECT * FROM Customer
WHERE customer_id IN (
    SELECT o.customer_id FROM Orders o
    INNER JOIN Product p ON o.product_id = p.product_id
    WHERE p.price > 20000
);
```

ALL — Condition must hold for every value in the subquery

Equivalent to comparing against the minimum (for <) or maximum (for >) of the subquery.

```
-- Products cheaper than ALL items bought by customer 2
SELECT * FROM Product
WHERE price < ALL (
    SELECT p.price FROM Product p
    INNER JOIN Orders o ON p.product_id = o.product_id
    WHERE o.customer_id = 2
);
```

ANY — Condition must hold for at least one value in the subquery

Equivalent to comparing against the minimum (for >) of the subquery result set.

```
-- Products more expensive than ANY item bought by customer 1
SELECT * FROM Product
WHERE price > ANY (
    SELECT p.price FROM Product p
    INNER JOIN Orders o ON p.product_id = o.product_id
```

```
    WHERE o.customer_id = 1
);
```

IS NULL / IS NOT NULL — Check for missing values

```
SELECT * FROM Customer WHERE city IS NULL;
SELECT * FROM Product  WHERE price IS NOT NULL;
```

NOT IN — Products not ordered by any customer

```
SELECT * FROM Product
WHERE product_id NOT IN (SELECT product_id FROM Orders);
```

Topic 7 — Stored Procedures & Functions with Cursors

Cursor Lifecycle — Always follow this order

Cursors iterate over a result set one row at a time inside a procedure or function.

```
DECLARE cur CURSOR FOR SELECT ...;                -- 1. Define the query
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;  -- 2. End-of-rows handler
OPEN cur;                                         -- 3. Execute the query
FETCH cur INTO var1, var2;                        -- 4. Get one row
CLOSE cur;                                        -- 5. Release resources
```

PROCEDURE — Named block executed with CALL; does not return a value

Example: calculate 10% bonus for every employee using a cursor.

```
DELIMITER //
CREATE PROCEDURE CalculateBonus()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE e_id INT;
    DECLARE e_salary DECIMAL(10,2);

    DECLARE cur CURSOR FOR SELECT emp_id, salary FROM employees;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    OPEN cur;
    read_loop: LOOP
        FETCH cur INTO e_id, e_salary;
        IF done THEN LEAVE read_loop; END IF;
        UPDATE employees SET bonus = e_salary * 0.10 WHERE emp_id = e_id;
    END LOOP;
    CLOSE cur;
END //
DELIMITER ;

CALL CalculateBonus();   -- execute the procedure
```

FUNCTION — Returns a value; used inside SELECT expressions

Example: count employees earning more than a given salary.

```
DELIMITER //
CREATE FUNCTION CountHighEarners(min_salary DECIMAL(10,2))
RETURNS INT DETERMINISTIC
BEGIN
    DECLARE done      INT DEFAULT FALSE;
    DECLARE count_val INT DEFAULT 0;
    DECLARE e_salary  DECIMAL(10,2);

    DECLARE cur CURSOR FOR SELECT salary FROM employees;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    OPEN cur;
    read_loop: LOOP
        FETCH cur INTO e_salary;
        IF done THEN LEAVE read_loop; END IF;
```

```
        IF e_salary > min_salary THEN
            SET count_val = count_val + 1;
        END IF;
    END LOOP;
    CLOSE cur;
    RETURN count_val;
END //
DELIMITER ;

SELECT CountHighEarners(50000) AS Employees_Above_50k;
```

Topic 8 — Triggers (Library System)

OLD vs NEW — Special row references inside triggers

OLD holds the value before the operation; NEW holds the new/incoming value.

```
OLD.column  -- value BEFORE the change  (available in UPDATE, DELETE)
NEW.column  -- value AFTER  the change  (available in INSERT, UPDATE)

-- In a BEFORE trigger you can modify NEW.column to change what gets written
-- In an AFTER  trigger OLD and NEW are read-only
```

AFTER UPDATE / AFTER DELETE — Log changes to an audit table

MySQL requires separate triggers for UPDATE and DELETE (no combined trigger syntax).

```
DELIMITER //
CREATE TRIGGER trg_library_update
AFTER UPDATE ON Library FOR EACH ROW
BEGIN
    INSERT INTO Library_Audit (B_id, Title, Edition, no_of_copies,
                               date_of_modification, type_of_operation, user)
    VALUES (OLD.B_id, OLD.Title, OLD.Edition, OLD.no_of_copies,
            CURDATE(), 'UPDATE', CURRENT_USER());
END; //

CREATE TRIGGER trg_library_delete
AFTER DELETE ON Library FOR EACH ROW
BEGIN
    INSERT INTO Library_Audit (B_id, Title, Edition, no_of_copies,
                               date_of_modification, type_of_operation, user)
    VALUES (OLD.B_id, OLD.Title, OLD.Edition, OLD.no_of_copies,
            CURDATE(), 'DELETE', CURRENT_USER());
END; //
DELIMITER ;
```

BEFORE INSERT — Validate / cap a value before the row is written

Used here to prevent issuing more copies than available in the library.

```
DELIMITER //
CREATE TRIGGER trg_check_available_copies
BEFORE INSERT ON Transaction FOR EACH ROW
BEGIN
    DECLARE available_copies INT;
    IF NEW.Issue_or_Return = 'Issue' THEN
        SELECT no_of_copies INTO available_copies
        FROM Library WHERE B_id = NEW.Book_id;

        IF NEW.no_of_copies > available_copies THEN
            SET NEW.no_of_copies = available_copies;  -- cap, don't reject
        END IF;
    END IF;
END; //
DELIMITER ;
```

AFTER INSERT — Auto-update another table based on the new row

Decrements library stock when a book is issued; increments it when returned.

```
DELIMITER //
CREATE TRIGGER trg_update_library_copies
AFTER INSERT ON Transaction FOR EACH ROW
BEGIN
    IF NEW.Issue_or_Return = 'Issue' THEN
        UPDATE Library SET no_of_copies = no_of_copies - NEW.no_of_copies
        WHERE B_id = NEW.Book_id;
    ELSEIF NEW.Issue_or_Return = 'Return' THEN
        UPDATE Library SET no_of_copies = no_of_copies + NEW.no_of_copies
        WHERE B_id = NEW.Book_id;
    END IF;
END; //
DELIMITER ;
```

Topic 9 — MongoDB: CRUD & Collection Operations

Insert

insertMany — Add multiple documents at once

```
use dbms_lab

db.Articles.insertMany([
  { Title: 'Intro to NoSQL', Author: 'Sharma', Author_age: 45,
    Dop: new Date('2023-01-15'), Category: 'IT',
    Comment: [{ Name: 'Ravi', Remarks: 'Great article!' }] },
  { Title: 'Global Elections 2024', Author: 'Verma', Author_age: 50,
    Dop: new Date('2023-05-10'), Category: 'Politics',
    Comment: [{ Name: 'Amit', Remarks: 'Interesting.' }] }
]);
```

Read / Find

findOne — Get the first document (optionally with a filter)

```
db.Articles.findOne();                        // first doc in collection
db.Articles.findOne({ Author: 'Sharma' });    // first doc matching filter
```

find with comparison operators

$gt = greater than, $gte = >=, $lt = less than, $lte = <=

```
// Authors older than 40; show only Author and Author_age, hide _id
db.Articles.find(
  { Author_age: { $gt: 40 } },
  { Author: 1, Author_age: 1, _id: 0 }
);

// Authors in age range 40-80
db.Articles.find(
  { Author_age: { $gte: 40, $lte: 80 } },
  { Author: 1, Author_age: 1, _id: 0 }
);
```

Update

updateOne with $set — Modify a field in the first matching document

```
db.Articles.updateOne(
  { Title: 'Intro to NoSQL', 'Comment.Name': 'Ravi' },
  { $set: { 'Comment.$.Remarks': 'Updated: Excellent read!' } }
);
```

updateOne with $push — Append an item to an array field

```
db.Articles.updateOne(
  { Title: 'Advanced MongoDB' },
  { $push: { Comment: { Name: 'Neha', Remarks: 'Helped me a lot.' } } }
```

```
);
```

updateMany with $set — Update a field in ALL matching documents

```
db.Articles.updateMany(
  { Category: 'IT' },
  { $set: { Category: 'Information Technology' } }
);
```

replaceOne with upsert — Replace entire doc; insert if not found

```
// Case 1: match found → entire document is replaced
db.Articles.replaceOne(
  { Title: 'Healthy Living' },
  { Title: 'Healthy Living', Author: 'Gupta', Category: 'Health and Wellness' },
  { upsert: true }
);

// Case 2: no match → new document is inserted
db.Articles.replaceOne(
  { Title: 'Future of AI' },
  { Title: 'Future of AI', Author: 'Singh', Category: 'Technology' },
  { upsert: true }
);
```

save() — Insert if no _id; replace if _id exists

```
// Without _id: behaves like insert
db.Articles.save({ Title: 'Healthy Living', Author: 'Gupta', Category: 'Health' });

// With _id: replaces the matching document
db.Articles.save({
  _id: ObjectId('652a1c000000000000000001'),
  Title: 'Finance 101 - Updated Edition', Author: 'Jain', Category: 'Finance'
});
```

Delete

deleteOne — Remove the first matching document

```
db.Articles.deleteOne({ Author: 'Sharma' });
```

deleteMany — Remove all matching documents

```
db.Articles.deleteMany({ Category: 'Politics' });
```

Topic 10 — MongoDB: Indexing & Aggregation Framework

Indexes

createIndex — Speed up queries; eliminates full collection scans

1 = ascending, -1 = descending. Without an index, MongoDB uses COLLSCAN (slow). With one, it uses IXSCAN (fast).

```
db.orders.createIndex({ cust_id: 1 });
db.orders.createIndex({ item_name: 1 });
db.orders.createIndex({ amount: 1 });
db.orders.createIndex({ status: 1 });
```

explain('executionStats') — Show whether index is used

```
// Without index: stage = COLLSCAN (scans every document)
db.orders.find({ status: 'P' }).explain('executionStats');

// After createIndex: stage = IXSCAN (index scan, much faster)
db.orders.createIndex({ status: 1 });
db.orders.find({ status: 'P' }).explain('executionStats');
```

distinct / countDocuments — Quick summary queries

```
db.orders.distinct('cust_id');              // unique customer IDs
db.orders.countDocuments({});               // total documents
db.orders.countDocuments({ status: 'P' });  // count matching filter
```

Aggregation Pipeline

$group — Group documents and compute aggregate values

_id sets the group key. $sum: 1 counts documents; $sum: '$field' totals a numeric field.

```
// Orders per customer
db.orders.aggregate([
  { $group: { _id: '$cust_id', total_orders: { $sum: 1 } } }
]);
```

$match → $group → $sort — Filter, aggregate, then sort

```
// Total pending amount per customer (descending)
db.orders.aggregate([
  { $match: { status: 'P' } },
  { $group: { _id: '$cust_id', pending_amount: { $sum: '$amount' } } },
  { $sort: { pending_amount: -1 } }
]);

// Total delivered amount per customer (ascending by ID)
db.orders.aggregate([
  { $match: { status: 'D' } },
  { $group: { _id: '$cust_id', delivered_amount: { $sum: '$amount' } } },
  { $sort: { _id: 1 } }
]);
```

$sort → $limit — Top N results

```
// Top 3 selling items by total quantity sold
db.orders.aggregate([
  { $group: { _id: '$item_name', total_sold: { $sum: '$quantity' } } },
  { $sort: { total_sold: -1 } },
  { $limit: 3 }
]);
```

$group with $sum: 1 — Date with most orders

```
db.orders.aggregate([
  { $group: { _id: '$DtOfOrder', count_orders: { $sum: 1 } } },
  { $sort: { count_orders: -1 } },
  { $limit: 1 }
]);
```

$min / $max — First and last order date per customer

```
db.orders.aggregate([
  { $group: {
      _id: '$cust_id',
      first_order: { $min: '$DtOfOrder' },
      last_order:  { $max: '$DtOfOrder' }
  }}
]);
```